

Integratieproject J3

Bordspelplatform

Inhoud

Inhoud	1
Inleiding	2
Organisatie van de teams	3
Applicatieontwikkeling en Software Management	3
Artificiële Intelligentie	3
Systeem- en Netwerkbeheer	3
Gedetailleerde vereisten	4
Applicatieontwikkeling	4
Functionele vereisten	4
Niet-functionele vereisten	5
Software management	6
Functionele vereisten	6
Niet-functionele vereisten	6
Artificiële intelligentie	8
Functionele Vereisten	8
1. Game State	8
2. AI-Modules	9
Deel 1 - AI Speler	9
Deel 2 - Chatbot voor het Platform	9
Optioneel Deel 3 - Machine Learning Model	9
Optioneel Deel 4 - Recommendation System	10
3. Data Logging	10
Niet-functionele Vereisten	10
Kwaliteit, Onderhoudbaarheid en Integratie	11
Systeem en netwerkbeheer	12

Inleiding

Het project omvat de ontwikkeling van een **digitaal platform** waarop spelers diverse **webgebaseerde bordspellen** kunnen spelen. Het platform is ontworpen met een sterke focus op gebruiksvriendelijkheid en stimuleert sociale interactie tussen spelers.

Een belangrijk uitgangspunt is de **open architectuur**, die het eenvoudig maakt om nieuwe spellen efficiënt te integreren.

Daarnaast worden **AI-gedreven functies** geïntegreerd die NPCs toevoegen en chatbot conversaties mogelijk maken.

Het platform wordt **in de cloud gehost**, met nadruk op **schaalbaarheid, beveiliging en betrouwbaarheid**.

Voor het management biedt het systeem **analysetools** waarmee gebruikspatronen inzichtelijk worden gemaakt.

Organisatie van de teams

Het geheel bestaat uit meerdere componenten die door verschillende teams worden ontwikkeld en geïntegreerd tot één werkend systeem.

Elk team werkt autonoom aan zijn onderdelen, maar de resultaten worden geïntegreerd tot één platform. Samenwerking tussen teams is essentieel en vereist een goede afstemming over API's, data, beveiliging, en deployment.

Applicatieontwikkeling en Software Management

IAO-studenten ontwikkelen de **gebruikersinterface, platform logica en 2 voorbeeldspellen**. Bovendien integreren ze één bestaand **extern spel** dat niet door henzelf ontwikkeld werd (dit wordt aangeleverd)

Sommige IAO-teams krijgen ondersteuning van een ISM-student, die **meewerkt aan het platform** en zich bijkomend richt op **analytische inzichten** via een ELK-stack.

Elk IAO(ISM)-team maakt via API's gebruik van de services van een AI-team.

Elk IAO(ISM)-team beheert zijn eigen CI/CD-pipeline(s) in GitLab.

Artificiële Intelligentie

IAI-studenten ontwikkelen **AI-functionaliteit voor het platform**. Ze werken onder andere aan AI-spelers die kunnen deelnemen aan de spelletjes, een chatbot en een aanbevelingssysteem.

De services van het IAI-team worden via API's door meerdere IAO(ISM)-teams gebruikt (geïndividualiseerd per IAO(ISM)-team waar nodig).

Elk IAI team beheert zijn eigen CI/CD-pipeline(s) in GitLab.

Systeem- en Netwerkbeheer

ISB-studenten zorgen voor een **stabiele, veilige en schaalbare infrastructuur**. Ze voorzien zowel een Kubernetes-cluster als een Docker compose omgeving waarop containers die door de IAO(ISM) en IAI teams worden aangemaakt kunnen draaien.

Elke ISB-team ondersteunt meerdere IAO(ISM) en IAI teams.

Gedetailleerde vereisten

Applicatieontwikkeling

Functionele vereisten

Het platform biedt spelers een geïntegreerde omgeving waarin ze verschillende bordspellen kunnen **ontdekken, aankopen en spelen**. Elk team ontwikkelt naast het platform ook **1 testspel (tic-tac-toe)** en **1 showcase spel** dat als showcase binnen het platform beschikbaar is. Daarnaast wordt een **extern spel** geïntegreerd dat door de opleiding wordt aangeleverd (en dus niet door het team zelf is ontwikkeld).

Het testspel is bedoeld om ervoor te zorgen dat de integratie tussen de teams soepel kan verlopen. Het showcase spel wordt pas in een 2e fase opgestart, na afstemming met het AI-team met betrekking tot de complexiteit van het spel (haalbaarheid van een AI-speler...). Denk hierbij in de richting van een bordspel waarbij **alle info beschikbaar** is voor de spelers (geen verborgen kaarten, ...) en dat **geen toevalsaspect** bevat (bv. dobbelstenen). De focus ligt dus eerder op de features en open architectuur van het platform dan op een complex bordspel.

Spelers kunnen hun **favoriete** bordspellen markeren, **vrienden** toevoegen en uitnodigen om samen te spelen, en **spelstatistieken** met elkaar vergelijken.

Een **lobby-systeem** zorgt voor een vlotte matchmaking tussen spelers die eenzelfde spel willen spelen.

Via een persoonlijk **profielbeheer** kunnen spelers hun gegevens, voorkeuren en prestaties opvolgen. Ze kunnen ervoor kiezen om een deel van hun profiel — zoals behaalde achievements of favoriete spellen — zichtbaar te maken voor alle spelers op het platform. Daarnaast kunnen vrienden elkaars profiel volledig bekijken, inclusief statistieken en recente activiteiten.

Een **achievement-systeem** registreert prestaties uit de verschillende spellen. Deze achievements worden door het platform beheerd en kunnen punten opleveren die inzetbaar zijn voor kleine platform voordelen.

Een **notificatie-systeem** verwittigt de gebruiker van belangrijke gebeurtenissen (aan zet, achievement behaald, friend request, ...). Notificatie kanalen (binnen het platform, email, ...) zijn instelbaar door de gebruiker in zijn profiel.

De look-and-feel van notificaties moet uniform zijn over alle spellen heen.

Binnen de spellen moet je kunnen **chatten** met de spelers die in het spel zitten. Omdat we niet dezelfde chatfunctionaliteit over alle spellen heen willen dupliceren, moet het chatten

gebeuren vanuit het platform. Het moet ook mogelijk zijn om verder te chatten buiten een spel, in het platform zelf.

Het platform wordt geïntegreerd met verschillende **AI-diensten** (zie verder).

Niet-functionele vereisten

Het platform en de games (tictactoe, showcase) worden ontwikkeld met een technologische stack, bestaande uit **React** voor de frontend, **Spring Boot** voor de backend, **RabbitMQ** voor messaging en **Keycloak** voor authenticatie en autorisatie. Een speler logt één keer in en kan daarna vrij tussen het platform en de verschillende games navigeren zonder opnieuw te moeten inloggen (SSO).

Er wordt veel aandacht besteed aan **codekwaliteit en onderhoudbaarheid**: de applicatie is modulair opgebouwd en voorzien van **unit- en integratietesten** voor zowel de spel- als platformlogica.

De concepten aangebracht in vakken zoals Software Architecture en User Interfaces 3 worden toegepast.

Realtime updates (spel, chat, ...) verlopen via **polling**. Er hoeft geen gebruik te worden gemaakt van web sockets,

Het ontwerp is **responsief**, met een duidelijke navigatie en een consistente gebruikerservaring.

De volledige ontwikkelcyclus wordt ondersteund door **CI/CD-pipelines** die de testen runnen en deploybare images bouwen.

Er worden **health endpoints** voor monitoring voorzien.

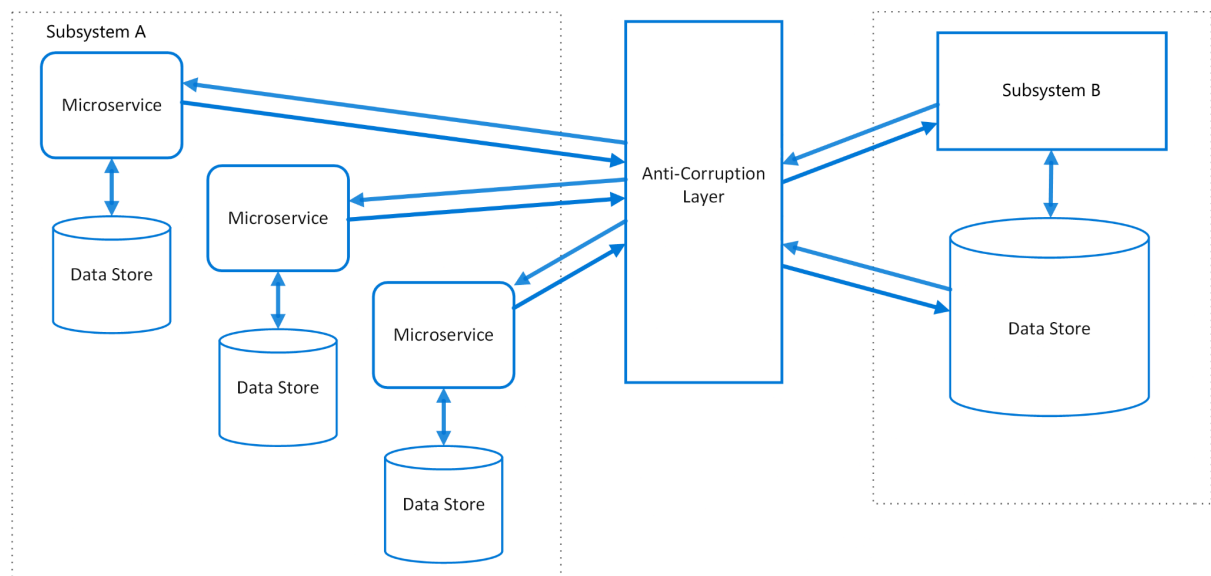
Zowel het platform als de bordspellen die beschikbaar zijn binnen het platform moeten apart gedeployed kunnen worden als eigen deployment unit.

De communicatie tussen het platform en de bordspellen verloopt via REST API en via messaging (RabbitMQ). Het is aan jullie om de inschatting te maken welke vorm van communicatie het meest aangewezen is. Houd hierbij rekening met het feit dat als een spel gestart is (vanuit de lobby van het platform) het mogelijk moet zijn om het spel te blijven spelen, zelfs als het platform ondertussen onbereikbaar is geworden.

Spellen moeten zich bij het opstarten kenbaar maken aan het platform, zodat het platform deze ook kan laten zien. Zij dienen hierbij alle info die nodig is (spel url, spel specifieke achievements, ...) mee te geven.

Als platform willen we zo veel mogelijk spellen aanbieden. Om dat mogelijk te maken, moeten andere developers makkelijk aan de slag kunnen gaan met de APIs. Om die reden moet er goede documentatie beschikbaar zijn.

Naast het zelf te ontwikkelen showcase spel wordt een bestaand **extern spel** geïntegreerd dat door de opleiding wordt aangeleverd. Dit spel heeft uiteraard geen kennis van hoe jullie platform eruit ziet. Daarom zal de verantwoordelijkheid bij jullie liggen om de events die dit spel genereert om te zetten naar events die jullie platform begrijpt. Jullie willen **niet** dat de events die het externe spel genereert tot in jullie domein komen, er zal dus een soort mapping/adapter laag tussen moeten zitten (cfr. anti corruption layer, een context mapping technique vanuit DDD). Hieronder een afbeelding dat dit showcaseset:



Het externe spel is hier “Subsystem B” en jullie platform en bijhorende modules is “Subsystem A”.

Software management

Functionele vereisten

Om de opbrengsten die voortvloeien uit het platform te optimaliseren heeft het managementteam **inzicht** nodig in het **gedrag van de gebruikers**. Hiervoor wordt een **afzonderlijke "Analytics platform"** opgezet.

Dit platform moet in staat zijn het management waardevolle inzichten te geven in het gebruikersgedrag op het platform **op basis waarvan ze de juiste strategische beslissingen kunnen nemen**. Een voorbeeld kan zijn dat ze de "evolutie van de opbrengsten per spel" kunnen tonen zodat ze kunnen beslissen voor welke spellen ze meer reclame willen maken. Er zijn heel veel zaken die je kan visualiseren. Inspiratie kan je onder andere vinden in [dit artikel](#).

Het management verwacht **verschillende thematische dashboards**. Elk dashboard focust dus op een andere categorie aan inzichten.

Daarnaast dient er **ook** een **dashboard** te worden gemaakt dat een **speler** inzicht geeft in de activiteiten op het platform. Denk hierbij goed na wat de gebruikers van het platform boeiende informatie vinden, maar ook welke informatie de activiteit en opbrengsten op het platform kan verhogen.

Niet-functionele vereisten

Je bouwt een "near real time" data processing pipeline waarbij je in de backend gegenereerde events naar de **RabbitMQ** message queue verstuurt. Deze **events** worden met behulp van **Logstash** opgeslagen in een **Elastic** database. De dashboards worden ontworpen in **Kibana**. Alle transformaties die nodig zijn voor je visualisaties, werk je uit in Elastic.

Samen met het ISB-team leg je vast wat er nodig is om de **Elastic Stack** correct te deployen.

Omdat het platform nog volop in ontwikkeling is, is er ook nood aan **gegenereerde data**. Zorg ervoor dat je op **elk dashboard grafieken** ziet **met realistische patronen**.

Je legt **samen met het applicatieteam** vast welke **informatie** er **aanwezig** moet zijn **in de events** die naar Elastic worden verstuurd. Voer hiervoor een **bronanalyse** uit op basis van de analysebehoeften.

Volgende zaken moeten **op git beschikbaar** worden gesteld: **configuraties, dashboard exports, transformatie-exports, code voor de generatie, wireframes en screenshots van de dashboard**.

Je databank en dashboardinstellingen moeten automatisch naar productie gebracht kunnen worden. Met een script moeten de in development ontwikkelde configuraties (index mappings, indexes, streams, aggregates, dashboards...) automatisch geïmporteerd worden in de productieomgeving van Elastic.

Artificiële intelligentie

De AI-functionaliteit van het platform bestaat uit verschillende onderling verbonden modules die samen de **intelligentielaag** van het systeem vormen. Elke module draagt bij aan een specifiek aspect van intelligent gedrag of gebruikersinteractie binnen het gaming-ecosysteem.

De **AI-speler** fungeert als een autonome agent die zelfstandig kan deelnemen aan bordspellen en strategische beslissingen neemt op basis van de spelstatus en regels.

Een **Chatbot** biedt een conversatie-interface die gebruikers ondersteunt bij het gebruik van het platform en spelregels op een toegankelijke manier uitlegt.

Tot slot voorzie je ook een **Recommendation Engine** die de betrokkenheid van speler vergroot door nieuwe of relevante spellen aan te bevelen op basis van hun speelgedrag en voorkeuren. Je probeert deze functionaliteit ook te integreren in de chatbot.

Samen zorgen al deze componenten ervoor dat het platform niet alleen **datagedreven** is, maar ook in staat is om te **leren**, zich **aan te passen** en **intelligent te interageren** met zijn gebruikers.

Functionele Vereisten

1. Game State

Voor de ontwikkeling van de AI-componenten bepaalt het team **welke game state** minimaal en optioneel bijgehouden moet worden. De game state bevat **alle informatie die nodig is om:**

1. de huidige spelsituatie eenduidig te beschrijven,
2. legale acties te bepalen o.b.v. zo'n toestand,
3. een manier om die state te evalueren,
4. een manier om de volgende toestand te genereren.

Wat kan er in de game state zitten?

Alle **observeerbare zaken**. Dus alle informatie die een gewone speler of een AI-speler kan zien:

- **Spelconfiguratie:** bordafmetingen, bordtoestand, locatie van de spelers, ...
- **Beurtinformatie:** wie is aan zet, beurt-/zetnummer, resterende tijd (als van toepassing).
- **Legale acties:** de set van acties die nu toegestaan zijn.
- **Geschiedenis (optioneel):** eerdere zetten
- **Spelstatus:** is het spel nog bezig, zo nee, wat is de uitkomst (win/verlies/remise) en bijbehorende **reward(s)**. Hoeveel punten zijn er al verdiend, ...

Denk na over de **serialisatie van de game state**: een eenduidige representatie (bijv. JSON) zodat de toestand kan worden opgeslagen en doorgestuurd worden over-the-wire.

Merk op!

Waak er als AI-team over dat er geen spellen gekozen worden die **niet-observeerbare/verborgen toestanden** hebben. Dit zijn toestanden die er wél zijn, maar niet zichtbaar voor tegenspelers (bijv. verborgen kaarten).

2. AI-Modules

Deel 1 - AI Speler

Ontwikkel een AI-component die zelfstandig kan deelnemen aan een bordspel binnen het platform. De AI-speler moet zetten kunnen bepalen op basis van de huidige bordstatus, spelregels en eventueel historische data.

De AI-speler gebruikt een of meerdere van de volgende algoritmen:

- **MCTS (Monte Carlo Tree Search)**
- **Minimax** (met of zonder Alpha-Beta pruning)
- **Heuristische benaderingen** of simulatie-gebaseerde beslissystemen
- **Reinforcement Learning** (optioneel, voor meer geavanceerde AI's)

De AI-speler communiceert met het platform via gestandaardiseerde **REST API's**, identiek aan menselijke spelers. De architectuur laat toe dat meerdere AI-spelers tegelijk actief zijn in verschillende sessies. Om de correcte werking van de AI-speler te monitoren, moeten er bepaalde **AI-prestaties** gelogd worden (zie [AI Gameplay Logging](#) voor meer informatie).

Deel 2 - Chatbot voor het Platform

Een chatbot kan ondersteuning bieden bij het spelen van de games op het platform. In principe is één chatbot voldoende voor alle games, maar uiteraard moet hij relevant informatie geven over het gekozen spel en niet over een ander spel dat niet geselecteerd is.

Database met Spelregels:

- De chatbot heeft toegang tot een database met de regels van alle beschikbare spellen.

- Bij gebruikersvragen (bijv. “Wat zijn de regels van Vier op een Rij?”) haalt de chatbot de relevante regels op.
- De regels worden **stap voor stap** weergegeven, zodat gebruikers eenvoudig begrijpen hoe het spel werkt.

Extra functionaliteit:

- De chatbot biedt ook **platformondersteuning** (bijv. uitleg over navigatie, inloggen, of instellingen).
- De chatbot maakt gebruik van **RAG (Retrieval-Augmented Generation)** en een **LLM** om relevante informatie dynamisch te genereren.

Deel 3 - Recommendation System

Een **aanbevelingsengine** stelt nieuwe spellen voor op basis van speelgeschiedenis en voorkeuren. Je integreert deze functionaliteit in de chatbot.

- **Collaborative filtering:** op basis van vergelijkbare gebruikers.
- **Content-based filtering:** op basis van kenmerken van eerder gespeelde spellen.

3. AI Gameplay Logging

Je wilt kunnen controleren of je AI-speler werkt en daarom moet je de **AI-prestaties** (zoals gekozen zetten, winpercentages, ...) visueel te kunnen volgen. Dit maakt het eenvoudiger om **trainingen, simulaties en spelgedrag** van AI-componenten te analyseren en te verbeteren.

Functionaliteiten:

- Loggen van data in real-time of batchgewijs (afhankelijk van de spelarchitectuur).
- Automatische logging van:
 - Spelstatussen en zetten
 - Duur van een spelbeurt
 - Win/verliesratio's van spelers en AI-spelers
 - AI-model (hyper)parameters
 - ... (kies zelf nuttig info)

Niet-functionele Vereisten

De AI-componenten worden ontwikkeld met een **Python-gebaseerde technologie-stack**, bestaande uit:

- **FastAPI, Flask of Django** voor het bouwen van RESTful API-services
- **Pandas, NumPy** en (indien van toepassing) scikit-learn, PyTorch of TensorFlow voor dataverwerking en AI-logica
- **PostgreSQL** of **MySQL** als relationele databank voor opslag van speler- en spelgegevens

- Docker voor **containerisatie** en GitLab CI/CD-pipelines voor build, test en deployment
- API-documentatie via **OpenAPI/Swagger**
- **TensorBoard** of alternatief voor het visualiseren van AI- en logdata.

Kwaliteit, Onderhoudbaarheid en Integratie

- Code is **modulair** opgebouwd en voorzien van **unit- en integratietesten**.
- Het ontwerp volgt principes uit **Software Architecture**
- AI-services communiceren met het platform via **REST API's**.
- De AI-services worden afzonderlijk beveiligd via **Keycloak**, waarbij elk **authenticatietoken** wordt gevalideerd voordat toegang wordt verleend.
- Elke AI-service wordt **afzonderlijk gedeployed** als eigen deployment unit op de infrastructuur van het SB-team.
- CI/CD-pipelines automatiseren het test-, build- en deploymentproces.
- Er wordt gezorgd voor duidelijke technische documentatie zodat AO-teams eenvoudig kunnen integreren met de AI-API's.
- Optioneel: Health-endpoints worden voorzien voor monitoring.

De volledige ontwikkelcyclus wordt ondersteund door **CI/CD-pipelines** die de testen runnen, de software bouwen en deze in productie brengen op de door het ISB-team aangeleverde infrastructuur.

Systeem en netwerkbeheer

De images worden aangemaakt door de IAO(ISM) en IAI teams via hun CI/CD pipelines.

Het registry waarlangs de images aangeboden worden, is af te spreken tussen de AO(SM)/AI- en ISB-teams

Er worden twee deployments voorzien:

- Kubernetes (op Google Cloud)
- lokaal via Docker Compose

De DNS-naam wordt vastgelegd door de ISB teams, in samenspraak met de AO(SM) en AI teams.

De volgende technologieën worden toegepast:

- voorziening van de clusters en opzetten van databases: Terraform
- deployment in Kubernetes (Google Cloud) en lokaal deployment (Docker Compose)
- monitoring

ISB-team werken met test images ('lege dozen') om vlot te kunnen starten zonder direct afhankelijk te zijn van geleverde containers. Stem hierover af met de teams die je ondersteunt.